PART A — FUNDAMENTALS

1. What is a software supply chain?
The software supply chain is everything involved in creating and delivering software: source code, third-party libraries and dependencies, build tools, CI/CD pipelines, repositories, and distribution channels.

2. What is a software dependency?
A dependency is a piece of software, such as a library, package, and framework, that an application relies on order to work. Developers reuse existing components instead of writing everything from scratch.

3. Explain a direct dependency.
A direct dependency is a component that developers add to their project and list in their own file.

4. Explain a transitive dependency.
A transitive dependency is a component pulled in indirectly because one of your direct dependencies needs it.

5. Why can outdated components create security risk?
Outdated components may contain publicly known vulnerabilities that have since been fixed in newer versions. Attackers can search for these known flaws and exploit them using ready-made tools, so a missing update leaves an easy entry point into the application.

6. Why can unmaintained components be a concern?
If a component is no longer maintained, its developers no longer release security patches. Any new vulnerability found in it stays unfixed, so the application stays exposed with no official remedy.

7. Why should software come from trusted sources?
Trusted official sources reduce the risk of tampered, malicious, or fake components. Packages from unknown websites can contain malware or backdoors that run with the application's privileges, so components should be verified.

8. Why should unused dependencies be removed?
Every dependency adds maintenance work it must be tracked, updated, and patched, so unused ones waste effort.

PART B — SBOM & VULNERABILITY MANAGEMENT

9. What does SBOM stand for?
- Software Bill of Materials

10. Explain an SBOM in your own words.
An SBOM is a detailed inventory of all the components inside a piece of software, including libraries, packages, and their versions.

11. Give three types of information an SBOM can help an organization understand.
    - Which components and dependencies are used in the software
    - Where each component came from and its license details.
    - Which versions of those components are used,[Mostly expired can be found]

12. Does an SBOM automatically fix vulnerabilities? Explain.
    - No. An SBOM only provides visibility.

13. What is a CVE?
A CVE (Common Vulnerabilities and Exposures) is a unique identifier given to a publicly disclosed security vulnerability.

14. Why should organizations monitor security advisories?
Advisories announce newly discovered vulnerabilities and available fixes. Monitoring them lets an organization learn about risks early, check whether its components are affected, and apply patches before attackers can exploit the flaw.

15. What is patch management?
Patch management is the process of identifying, evaluating, testing, and applying software updates (patches) to fix vulnerabilities and bugs, in a controlled and regular way.

16. Why should updates be tested before production deployment?
Updates can introduce bugs or compatibility problems that break features or cause downtime in the live system. So it makes it easy to finish it.

PART C — CI/CD SECURITY

17. What is CI/CD at a beginner level?
CI/CD stands for Continuous Integration / Continuous Delivery (or Deployment). It is an automated pipeline that takes code changes, then builds, tests, packages, and deploys the software.

18. Why is CI/CD part of the software supply chain?
The CI/CD pipeline sits between the source code and the finished product. It pulls in dependencies, builds the software, and delivers it to users.

19. Explain Least Privilege for a CI/CD system.
Least privilege means giving the pipeline, and the accounts and tokens it uses, only the minimum access needed to do its job, and nothing more.

20. Why should production secrets not be stored openly in source-code repositories?
Secrets such as passwords, API keys, and tokens stored in a repository can be seen by anyone with access to it, and they remain in the commit history even after deletion. If the repository is leaked or made public, attackers can use these secrets to access production systems and data. Secrets should instead be kept in a secrets manager or protected environment variables, with access controlled and rotated regularly.

PART D — PRACTICAL ANALYSIS

Review:

legacy-report-parser
Status: No Longer Maintained

old-ui-helper
Status: Outdated
Used: No

auth-library
Status: Maintained
Transitive Dependency: legacy-token-helper
Transitive Status: Unsupported

random-file-tool
Source: Unknown Website

Answer:

1. Which component is unmaintained: legacy-report-parser

2. Which unused component should be considered for removal: old-ui-helper

3. Which component contains an unsupported transitive dependency: auth-library

4. Name that transitive dependency: legacy-token-helper

5. Which component came from an unknown source: random-file-tool

6. Why is that a security concern?
A component from an unknown website cannot be verified as authentic or untampered. It may contain malware, a backdoor, or hidden vulnerabilities.

7. How could an SBOM help?
- Reveal hidden components, such as legacy-token-helper, which is not listed directly but is pulled in by auth-library.
- Respond faster to new vulnerabilities, because the team can check the inventory to see whether it is affected and then decide what to patch, replace, or remove.

PART E — CI/CD CASE

Fictional configuration:

Repository MFA:
Enabled

Build Server Admin Access:
All Developers

Production Secret:
Stored in Repository

Dependency Inventory:
Not Maintained

Updates:
Deployed Directly to Production Without Testing

Answer:

1. Identify the positive security control: Repository MFA

2. Identify the four main security concerns.
- Admin access to all developers lead excessive privilege.
- Company may not be able to detect outdated dependencies.
- Changes released without proper testing process.
- Untested updates comes with security vulnerabilities.

3. Give four defensive recommendations.
- Apply the Principle of Least Privilege.
- Maintain a Dependency Inventory.
- Implement Testing Before Production Deployment.
- Secure Production Secrets.